1. vehicle_handlers.go

```
package handlers

import (
"github.com/Automobile-Association-of-Kenya/aa-core-system/internal/vehicle-service/services"
"github.com/Automobile-Association-of-Kenya/aa-core-system/pkg/response"
"github.com/gin-gonic/gin"
"github.com/go-playground/validator/v10"
)

type VehicleHandler struct {
vehicleService *services.VehicleService
validator *validator.Validate
}

func NewVehicleHandler(vehicleService *services.VehicleService) *VehicleHandler {
return &VehicleHandler{
vehicleService: vehicleService,
validator: validator.New(),
}
}

// Make handlers
func (h *VehicleHandler) CreateMake(c *gin.Context) {
var req services.CreateMakeRequest
if err := c.ShouldBindJSON(&req); err != nil {
response.BadRequest(c, "Invalid request body")
return
}

if err := h.validator.Struct(req); err != nil {
response.ValidationError(c, err.Error())
return
}

make, err := h.vehicleService.CreateMake(req)
if err != nil {
response.BadRequest(c, err.Error())
return
}

response.Success(c, "Vehicle make created successfully", make)
}
```

```
func (h *VehicleHandler) GetAllMakes(c *gin.Context) {
makes, err := h.vehicleService.GetAllMakes()
if err != nil {
response.InternalServerError(c, "Failed to retrieve vehicle makes")
return
}

response.Success(c, "Vehicle makes retrieved successfully", makes)
}

//func (h *VehicleHandler) GetMakeByID(c *gin.Context) {
// makeID := c.Param("id")
//
// make, err := h.vehicleService.GetMakeByID(makeID)
// if err != nil {
// response.NotFound(c, "Vehicle make not found")
// return
// }
//
// response.Success(c, "Vehicle make retrieved successfully", make)
//}

// Model handlers
func (h *VehicleHandler) CreateModel(c *gin.Context) {
var req services.CreateModelRequest
if err := c.ShouldBindJSON(&req); err != nil {
response.BadRequest(c, "Invalid request body")
return
}

if err := h.validator.Struct(req); err != nil {
response.ValidationError(c, err.Error())
return
}

model, err := h.vehicleService.CreateModel(req)
if err != nil {
response.BadRequest(c, err.Error())
return
}

response.Success(c, "Vehicle model created successfully", model)
}
```

```
//func (h *VehicleHandler) GetAllModels(c *gin.Context) {
// models, err := h.vehicleService.GetAllModels()
// if err != nil {
// response.InternalServerError(c, "Failed to retrieve vehicle models")
// return
// }
//
// response.Success(c, "Vehicle models retrieved successfully", models)
//}

func (h *VehicleHandler) GetModelsByMake(c *gin.Context) {
makeID := c.Param("make_id")

models, err := h.vehicleService.GetModelsByMake(makeID)
if err != nil {
response.InternalServerError(c, "Failed to retrieve vehicle models")
return
}

response.Success(c, "Vehicle models retrieved successfully", models)
}

// Customer Vehicle handlers
func (h *VehicleHandler) CreateCustomerVehicle(c *gin.Context) {
var req services.CreateCustomerVehicleRequest
if err := c.ShouldBindJSON(&req); err != nil {
response.BadRequest(c, "Invalid request body")
return
}

if err := h.validator.Struct(req); err != nil {
response.ValidationError(c, err.Error())
return
}

// Authorization check for customers
userID := c.GetString("user_id")
userType := c.GetString("user_type")

if userType == "customer" && req.CustomerID != userID {
response.Forbidden(c, "Cannot create vehicle for another customer")
return
}
```

```
vehicle, err := h.vehicleService.CreateCustomerVehicle(req)
if err != nil {
response.BadRequest(c, err.Error())
return
}

response.Success(c, "Customer vehicle created successfully", vehicle)
}

func (h *VehicleHandler) GetCustomerVehicles(c *gin.Context) {
customerID := c.Param("customer_id")

// Authorization check
userID := c.GetString("user_id")
userType := c.GetString("user_type")

if userType == "customer" && customerID != userID {
response.Forbidden(c, "Cannot access another customer's vehicles")
return
}

vehicles, err := h.vehicleService.GetCustomerVehicles(customerID)
if err != nil {
response.InternalServerError(c, "Failed to retrieve customer vehicles")
return
}

response.Success(c, "Customer vehicles retrieved successfully", vehicles)
}

//func (h *VehicleHandler) GetVehicleByID(c *gin.Context) {
// vehicleID := c.Param("id")
//
// vehicle, err := h.vehicleService.GetCustomerVehicles(vehicleID)
// if err != nil {
// response.NotFound(c, "Vehicle not found")
// return
// }
//
// // Authorization check
// userID := c.GetString("user_id")
// userType := c.GetString("user_type")
//
```

```
// if userType == "customer" && vehicle.CustomerID != userID {
// response.Forbidden(c, "Cannot access another customer's vehicle")
// return
// }
//
// response.Success(c, "Vehicle retrieved successfully", vehicle)
//}

func (h *VehicleHandler) GetVehicleByRegistration(c *gin.Context) {
registrationNumber := c.Param("registration")

vehicle, err := h.vehicleService.GetVehicleByRegistration(registrationNumber)
if err != nil {
response.NotFound(c, "Vehicle not found")
return
}

response.Success(c, "Vehicle retrieved successfully", vehicle)
}
```

2. vehicle_service.go

```
package services

import (
"fmt"
"time"

"github.com/Automobile-Association-of-Kenya/aa-core-system/internal/shared/kafka"
"github.com/Automobile-Association-of-Kenya/aa-core-system/internal/vehicle-
service/models"
"github.com/Automobile-Association-of-Kenya/aa-core-system/internal/vehicle-
service/repositories"
"github.com/google/uuid"
)

type VehicleService struct {
vehicleRepo *repositories.VehicleRepository
kafkaProducer *kafka.Producer
}

func NewVehicleService(vehicleRepo *repositories.VehicleRepository, kafkaProducer
*kafka.Producer) *VehicleService {
return &VehicleService{
```

```
vehicleRepo: vehicleRepo,
kafkaProducer: kafkaProducer,
}
}

// Customer Vehicle operations
type CreateCustomerVehicleRequest struct {
CustomerID string `json:"customer_id" validate:"required"`
MakeID string `json:"make_id" validate:"required"`
ModelID string `json:"model_id" validate:"required"`
RegistrationNo string `json:"registration_no" validate:"required"`
YearOfManufacture int `json:"year_of_manufacture"
validate:"required,min=1900,max=2030"`
Color string `json:"color"`
ChassisNumber *string `json:"chassis_number,omitempty"`
EngineNumber *string `json:"engine_number,omitempty"`
FuelType models.FuelType `json:"fuel_type" validate:"required"` // FIXED: Now
available in models
Transmission models.TransmissionType `json:"transmission" validate:"required"` //
FIXED: Now available in models
EngineCapacity *float64 `json:"engine_capacity,omitempty"`
Mileage *int `json:"mileage,omitempty"`
PurchaseDate *time.Time `json:"purchase_date,omitempty"`
Notes *string `json:"notes,omitempty"`
}

func (s *VehicleService) CreateCustomerVehicle(req CreateCustomerVehicleRequest)
(*models.CustomerVehicle, error) {
// Verify customer exists
_, err := s.vehicleRepo.GetCustomerByID(req.CustomerID)
if err != nil {
return nil, fmt.Errorf("customer not found: %v", err)
}

// Verify make exists
_, err = s.vehicleRepo.GetMakeByID(req.MakeID)
if err != nil {
return nil, fmt.Errorf("vehicle make not found: %v", err)
}

// Verify model exists
model, err := s.vehicleRepo.GetModelByID(req.ModelID)
if err != nil {
return nil, fmt.Errorf("vehicle model not found: %v", err)
```

```
}

// Verify model belongs to make
if model.MakeID != req.MakeID {
return nil, fmt.Errorf("model does not belong to the specified make")
}

// Check if registration number already exists
existing, err := s.vehicleRepo.GetVehicleByRegistration(req.RegistrationNo)
if err == nil && existing != nil {
return nil, fmt.Errorf("vehicle with registration number %s already exists",
req.RegistrationNo)
}

// Create vehicle
vehicle := &models.CustomerVehicle{
CustomerID: req.CustomerID,
MakeID: req.MakeID,
ModelID: req.ModelID,
RegistrationNo: req.RegistrationNo,
YearOfManufacture: req.YearOfManufacture,
Color: req.Color,
ChassisNumber: req.ChassisNumber,
EngineNumber: req.EngineNumber,
FuelType: req.FuelType, // FIXED: Now works
Transmission: req.Transmission, // FIXED: Now works
EngineCapacity: req.EngineCapacity,
Mileage: req.Mileage,
PurchaseDate: req.PurchaseDate,
Notes: req.Notes,
Status: models.VehicleStatusActive,
}

if err := s.vehicleRepo.CreateCustomerVehicle(vehicle); err != nil {
return nil, fmt.Errorf("failed to create vehicle: %v", err)
}

// Publish vehicle created event
s.publishVehicleEvent("vehicle.created", vehicle)

return vehicle, nil
}

// Event handlers for customer sync
```

```
func (s *VehicleService) HandleCustomerCreated(event kafka.Event) error {
customerData := event.Data

customer := &models.Customer{ // FIXED: Now using local Customer model
ID: customerData["user_id"].(string),
Email: customerData["email"].(string),
Phone: customerData["phone"].(string),
FirstName: customerData["first_name"].(string),
LastName: customerData["last_name"].(string),
Status: "pending_verification",
}

if err := s.vehicleRepo.CreateOrUpdateCustomer(customer); err != nil {
return fmt.Errorf("failed to create customer in vehicle service: %v", err)
}

return nil
}

func (s *VehicleService) HandleCustomerVerified(event kafka.Event) error {
customerID := event.Data["user_id"].(string)

customer, err := s.vehicleRepo.GetCustomerByID(customerID)
if err != nil {
return fmt.Errorf("customer not found in vehicle service: %v", err)
}

customer.Status = "active"
return s.vehicleRepo.CreateOrUpdateCustomer(customer)
}

// Get methods
func (s *VehicleService) GetCustomerVehicles(customerID string)
([]models.CustomerVehicle, error) {
return s.vehicleRepo.GetCustomerVehicles(customerID)
}

func (s *VehicleService) GetVehicleByID(vehicleID string) (*models.CustomerVehicle,
error) {
return s.vehicleRepo.GetVehicleByID(vehicleID)
}

func (s *VehicleService) GetVehicleByRegistration(regNo string)
(*models.CustomerVehicle, error) {
```

```
return s.vehicleRepo.GetVehicleByRegistration(regNo)
}

// Vehicle Make operations
type CreateMakeRequest struct {
Name string `json:"name" validate:"required"`
Country string `json:"country"`
LogoURL *string `json:"logo_url,omitempty"`
}

func (s *VehicleService) CreateMake(req CreateMakeRequest) (*models.VehicleMake,
error) {
make := &models.VehicleMake{
Name: req.Name,
Country: req.Country,
LogoURL: req.LogoURL,
}

if err := s.vehicleRepo.CreateMake(make); err != nil {
return nil, fmt.Errorf("failed to create make: %v", err)
}

return make, nil
}

func (s *VehicleService) GetAllMakes() ([]models.VehicleMake, error) {
return s.vehicleRepo.GetAllMakes()
}

// Vehicle Model operations
type CreateModelRequest struct {
MakeID string `json:"make_id" validate:"required"`
Name string `json:"name" validate:"required"`
Description *string `json:"description,omitempty"` // FIXED: Now available
VehicleType models.VehicleType `json:"vehicle_type" validate:"required"`
StartYear int `json:"start_year" validate:"required"`
EndYear *int `json:"end_year,omitempty"`
}

func (s *VehicleService) CreateModel(req CreateModelRequest)
(*models.VehicleModel, error) {
// Verify make exists
_, err := s.vehicleRepo.GetMakeByID(req.MakeID)
if err != nil {
```

```
return nil, fmt.Errorf("vehicle make not found: %v", err)
}

model := &models.VehicleModel{
MakeID: req.MakeID,
Name: req.Name,
Description: req.Description, // FIXED: Now works
VehicleType: req.VehicleType,
StartYear: req.StartYear,
EndYear: req.EndYear,
}

if err := s.vehicleRepo.CreateModel(model); err != nil {
return nil, fmt.Errorf("failed to create model: %v", err)
}

return model, nil
}

func (s *VehicleService) GetModelsByMake(makeID string) ([]models.VehicleModel,
error) {
return s.vehicleRepo.GetModelsByMake(makeID)
}

// Utility methods
func (s *VehicleService) SearchVehicles(query string, limit, offset int)
([]models.CustomerVehicle, error) {
return s.vehicleRepo.SearchVehicles(query, limit, offset)
}

func (s *VehicleService) publishVehicleEvent(eventType string, vehicle
*models.CustomerVehicle) {
if s.kafkaProducer == nil {
return
}

event := kafka.Event{
ID: uuid.New().String(),
Type: eventType,
Source: "vehicle-service",
Data: map[string]interface{}{
"vehicle_id": vehicle.ID,
"customer_id": vehicle.CustomerID,
"registration_no": vehicle.RegistrationNo,
```

```
"make_id": vehicle.MakeID,
"model_id": vehicle.ModelID,
"year": vehicle.YearOfManufacture,
"fuel_type": string(vehicle.FuelType),
"transmission": string(vehicle.Transmission),
},
Timestamp: time.Now().Unix(),
Version: "1.0",
}

s.kafkaProducer.PublishEvent("vehicle_events", event)
}
```

3. vehicle_repositories.go

```
package repositories

import (
"github.com/Automobile-Association-of-Kenya/aa-core-system/internal/vehicle-
service/models"
"gorm.io/gorm"
)

type VehicleRepository struct {
db *gorm.DB
}

func NewVehicleRepository(db *gorm.DB) *VehicleRepository {
return &VehicleRepository{db: db}
}

// Customer operations (synced from user service)
func (r *VehicleRepository) CreateOrUpdateCustomer(customer *models.Customer)
error {
var existingCustomer models.Customer
err := r.db.Where("id = ?", customer.ID).First(&existingCustomer).Error

if err == gorm.ErrRecordNotFound {
return r.db.Create(customer).Error
}

return r.db.Save(customer).Error
}
```

```
func (r *VehicleRepository) GetCustomerByID(customerID string) (*models.Customer,
error) {
var customer models.Customer
err := r.db.First(&customer, "id = ?", customerID).Error
return &customer, err
}

// Vehicle Make operations
func (r *VehicleRepository) CreateMake(make *models.VehicleMake) error {
return r.db.Create(make).Error
}

func (r *VehicleRepository) GetAllMakes() ([]models.VehicleMake, error) {
var makes []models.VehicleMake
err := r.db.Where("is_active = ?", true).Order("name").Find(&makes).Error
return makes, err
}

func (r *VehicleRepository) GetMakeByID(makeID string) (*models.VehicleMake,
error) {
var make models.VehicleMake
err := r.db.Preload("Models").First(&make, "id = ?", makeID).Error
return &make, err
}

func (r *VehicleRepository) UpdateMake(make *models.VehicleMake) error {
return r.db.Save(make).Error
}

// Vehicle Model operations
func (r *VehicleRepository) CreateModel(model *models.VehicleModel) error {
return r.db.Create(model).Error
}

func (r *VehicleRepository) GetModelsByMake(makeID string)
([]models.VehicleModel, error) {
var models []models.VehicleModel
err := r.db.Where("make_id = ? AND is_active = ?", makeID,
true).Order("name").Find(&models).Error
return models, err
}

func (r *VehicleRepository) GetModelByID(modelID string) (*models.VehicleModel,
error) {
```

```
var model models.VehicleModel
err := r.db.Preload("Make").First(&model, "id = ?", modelID).Error
return &model, err
}

func (r *VehicleRepository) UpdateModel(model *models.VehicleModel) error {
return r.db.Save(model).Error
}

// Customer Vehicle operations
func (r *VehicleRepository) CreateCustomerVehicle(vehicle *models.CustomerVehicle)
error {
return r.db.Create(vehicle).Error
}

func (r *VehicleRepository) GetCustomerVehicles(customerID string)
([]models.CustomerVehicle, error) {
var vehicles []models.CustomerVehicle
err := r.db.Preload("Make").Preload("Model").Where("customer_id = ?",
customerID).Find(&vehicles).Error
return vehicles, err
}

func (r *VehicleRepository) GetVehicleByID(vehicleID string)
(*models.CustomerVehicle, error) {
var vehicle models.CustomerVehicle
err := r.db.Preload("Customer").Preload("Make").Preload("Model").First(&vehicle, "id
= ?", vehicleID).Error
return &vehicle, err
}

func (r *VehicleRepository) GetVehicleByRegistration(regNo string)
(*models.CustomerVehicle, error) {
var vehicle models.CustomerVehicle
err := r.db.Preload("Customer").Preload("Make").Preload("Model").First(&vehicle,
"registration_no = ?", regNo).Error
return &vehicle, err
}

func (r *VehicleRepository) UpdateCustomerVehicle(vehicle
*models.CustomerVehicle) error {
return r.db.Save(vehicle).Error
}
```

```
func (r *VehicleRepository) DeleteCustomerVehicle(vehicleID string) error {
return r.db.Delete(&models.CustomerVehicle{}, "id = ?", vehicleID).Error
}

// Vehicle Image operations
func (r *VehicleRepository) CreateVehicleImage(image *models.VehicleImage) error {
return r.db.Create(image).Error
}

func (r *VehicleRepository) GetVehicleImages(vehicleID string)
([]models.VehicleImage, error) {
var images []models.VehicleImage
err := r.db.Where("vehicle_id = ?", vehicleID).Order("is_primary DESC, created_at
ASC").Find(&images).Error
return images, err
}

func (r *VehicleRepository) DeleteVehicleImage(imageID string) error {
return r.db.Delete(&models.VehicleImage{}, "id = ?", imageID).Error
}

// Search operations
func (r *VehicleRepository) SearchVehicles(query string, limit, offset int)
([]models.CustomerVehicle, error) {
var vehicles []models.CustomerVehicle
err := r.db.Preload("Customer").Preload("Make").Preload("Model").
Where("registration_no ILIKE ? OR chassis_number ILIKE ? OR engine_number
ILIKE ?",
"%"+query+"%", "%"+query+"%", "%"+query+"%").
Limit(limit).Offset(offset).Find(&vehicles).Error
return vehicles, err
}
```